

Network Programming HW2 Report: Dual Real-Time Tetris Battle

Zheng Wu Qian (112550200)

1 System Architecture

The system is implemented as a Client-Server model with three distinct, self-contained service layers, communicating exclusively via TCP sockets.

1.1 Database Server (DB Server)

- **Role:** Provides persistence and standardized CRUD/Query operations for all application data (`User`, `Room`, `GameLog`).
- **Implementation:** A standalone Python process (`db_server.py`) listening on a fixed TCP Port **10001**.
- **Persistence:** Uses SQLite internally. Crucially, all data access, including by the Lobby Server, must go through the DB Server's Socket API, ensuring separation of the persistence layer.
- **Communication:** Uses the Length-Prefixed Framing Protocol with JSON payload.

1.2 Central Lobby Server (Lobby Server)

- **Role:** Manages user sessions, room creation, matchmaking, and game launch orchestration.
- **Implementation:** `lobby_server.py` runs on a fixed TCP Port **10002**. It uses threading to handle concurrent client connections.
- **Game Launch:** When a match starts, the Lobby Server dynamically launches a separate `game_server.py` process on a port allocated from the range **10100–10200** and relays this connection info to the players.
- **DB Interaction:** It acts as a client to the DB Server to handle registration, login, and game log reporting.

1.3 Game Server

- **Role:** Serves as the central authority for a single 2-player match.
- **Architecture:** A dedicated process (`game_server.py`) dynamically instantiated by the Lobby Server.
- **Server Authority:** It maintains the current state of both players' Tetris boards, processes all game inputs (`INPUT`) from clients, applies gravity, determines line clears, scores, and broadcasts game state snapshots (`SNAPSHOT`). The clients are 'thin,' only responsible for sending key presses and rendering the received state.

2 Communication Protocol

2.1 Length-Prefixed Framing Protocol

All TCP communication within the system (`DB ↔ Lobby ↔ Client ↔ Game Server`) adheres to the mandatory Length-Prefixed Framing Protocol, implemented in `protocol.py`.

- **Format:** `[4-byte length] [JSON body]`
- **Header:** A 4-byte unsigned integer (`uint32`) packed using `struct.pack('!I', length)`, ensuring **Network Byte Order** (Big-Endian). This value indicates the exact byte size of the subsequent JSON body.
- **Message Body:** UTF-8 encoded JSON string, containing the request/response data.
- **Partial I/O:** Helper functions `_recv_all` and `_send_all` handle partial reads/writes, ensuring the full 4-byte header and the entire message body are completely transmitted/received, mitigating TCP streaming issues.
- **Size Limit:** Messages exceeding 64 KiB (`MAX_MESSAGE_LENGTH = 65536`) are rejected, preventing resource abuse.

2.2 Lobby Server Event Flow (Example)

Lobby operations follow a request-response pattern using JSON payloads.

```
1 {  
2   "action": "start_game",  
3   "data": {}  
4 }
```

Listing 1: Start Game Request (Client → Lobby)

```
1 {  
2   "success": true,  
3   "gamePort": 10100,  
4   "players": [1, 2],  
5   "playerNames": ["playerA", "playerB"]  
6 }
```

Listing 2: Game Start Response (Lobby → Client)

3 Gameplay and Synchronization Strategy

3.1 Game Rules and Termination

- **Game Type:** Two-player real-time Tetris (implemented in `tetris_logic.py`).
- **Grid Size:** 10×20 .
- **Scoring:** Standard scoring based on line clears (100, 300, 500, 800 for 1-4 lines, multiplied by the current level). Level increases every 10 lines cleared.
- **Game Over:** When a player's piece locks above the visible playfield (topping out).
- **Termination Condition:** The game uses a **Survival Match** rule. The match ends when one player tops out, and the **last player surviving** (or the player with the higher score if both top out simultaneously/disconnect) is declared the winner.

3.2 Synchronization and Latency Mitigation

The core synchronization mechanism relies on the Server Authority model combined with initial consistency seeding.

1. **Seed Consistency:** Upon game start, the Game Server generates and broadcasts a random `seed` and `bagRule` (7-bag randomizer) to both clients in the `WELCOME` message. The server uses this same seed to initialize its two `TetrisGame` logic instances, ensuring the sequence of falling pieces is identical for both players.
2. **Input Event (Client $\rightarrow$ Server):** Players send granular action events (e.g., `LEFT`, `CW`, `HARD_DROP`) asynchronously via `INPUT` messages. The client's GUI is updated only after receiving a `SNAPSHOT` confirming the move.
3. **Snapshot State (Server $\rightarrow$ Clients):** The Game Server's loop (running at a fixed rate for gravity updates) checks for inputs, updates the game state, and broadcasts a `SNAPSHOT` message to all players and spectators.
4. **Board Compression:** The board state uses a simple compression (`boardRLE`) where each row is a string of cell IDs separated by a vertical pipe (`|`), minimizing message size.

4 Game Flow and Reporting

4.1 Game Client Auto-Launch

The `lobby_client.py` runs a background thread that periodically checks the room status with the Lobby Server. When the status changes to `playing`, the client automatically launches a separate `game_client.py` process as a subprocess, passing the necessary host, port, and user credentials.

4.2 Handling Disconnections and Exceptions

- **Lobby Disconnection:** If a user disconnects from the Lobby, they are automatically logged out, removed from the `online_users` set, and forced to leave any room they were in, preventing duplicate login issues and cleaning up server state.
- **Game Server Disconnection:** If a player disconnects mid-game, the Game Server detects the broken socket and triggers an immediate game end (`end_game_insufficient_players`) declaring the remaining player as the winner and initiating server shutdown.
- **Invalid Input/Protocol:** `protocol.py` validates the 4-byte header length. Game Server ignores invalid inputs, ensuring the game loop remains stable.

4.3 Game End Reporting

1. **Game Server Action:** When the game ends (due to top-out or disconnection), the Game Server calculates final scores and winner, broadcasts the `GAME_END` message to all clients/spectators.
2. **Lobby Notification:** The Game Server then acts as a client to the Lobby Server to send a final `game_ended` request, including the results and the `startAt` timestamp.
3. **DB Update and Cleanup:** The Lobby Server receives this report, updates the room status in the DB to `idle`, creates a permanent entry in the `GameLog` collection, and finally terminates the Game Server process.

5 Spectator Mode (Optional Feature)

The system supports a fully functional Spectator Mode:

- **Access:** The client requests the Game Server port for a room with status `playing` via the `spectate_room` API on the Lobby Server.
- **Game Server Role:** The Spectator Client connects with the `spectate: true` flag in the `HELLO` message. The Game Server adds them to the `spectators` list, but they are prevented from sending `INPUT` messages.
- **Client View:** The Spectator Client renders both players' boards side-by-side using the received `SNAPSHOT` data, with their UI adjusted to read-only mode.